

slot-c

34 B

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 1 – ANALYTICAL PROBLEM SOLVING

Course Code:	CSA1402	Course Title:	COMPILER DESIGN
CO Assessed:	CO1 – Imitation (L1)	Assessment:	Assessment Tool 1 – ANALYTICAL PROBLEM SOLVING
Weightage:	40% of CIA		
CO1 Statement:	Apply lexical analysis for token specification and recognition using LEX tool. (BL3)		
Student Name:	D K SUMALATHA	Reg. No.:	192424023
Section / Batch:		Date:	16/July/26

Code of Conduct

(192424023)

I, D.K. Sumalatha (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student:

D.K. Sumalatha**Instructions**

- Develop a modular and efficient Python program that satisfies all the functional requirements specified in the problem statement.
- Demonstrate the correctness of the implementation using suitable test cases and justify the output obtained.
- Follow good programming practices, including meaningful variable names, proper code indentation, comments, and error handling wherever applicable.
- Duration: 30 minutes.

Q. No	Problem Understanding (20)	Method Selection (20)	Solution Process (25)	Accuracy of Calculation (20)	Interpretation of Results (15)	Total Score (out of 100)
1	4/4	3/4	4/4	4/4	3/4	91.25
2	4/4	4/4	3/4	4/4	4/4	93.75
3	4/4	4/4	3/4	4/4	4/4	93.75
4	4/4	4/4	3/4	3/4	4/4	88.75
5	3/4	3/4	4/4	4/4	4/4	90
OVERALL RUBRICS VALUE						
	/4	/4	/4	/4	/4	91.5
	/20	/20	/25	/20	/15	/100

N.D.H



Analytical problem solving.

Compiler Phases:-

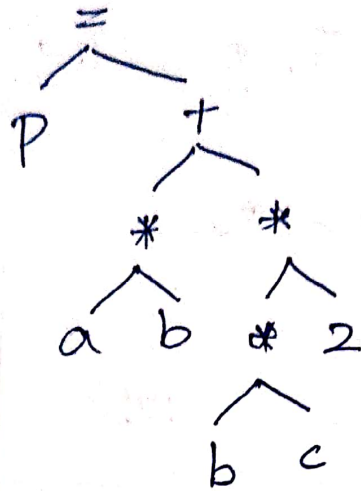
Given $P = (a * b) + (b * c) * 2$

a) Lexical Analysis (Tokens)

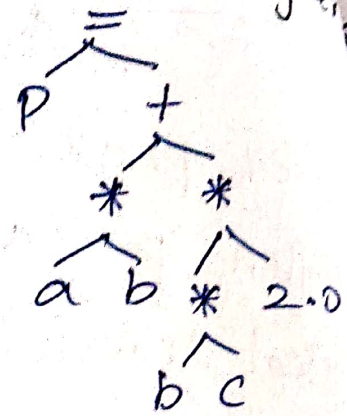
Token	Type
P	Identifier
=	Assignment operator
(	Left parenthesis
a	Identifier
*	Multiplication operator
b	Identifier
)	Right parenthesis
+	Addition operator
(	Left parenthesis
b	Identifier
)	Right parenthesis
*	Multiplication operator
2	constant

Total Tokens = 15

b) Syntax Tree



c) Semantic Analyzer



d) Intermediate code (Three Address code).

$\text{temp}_1 = a * b$

$\text{temp}_2 = b * c$

$\text{temp}_3 = \text{temp}_2 * 2$

$\text{temp}_4 = \text{temp}_1 + \text{temp}_3$

$P = \text{temp}_4$

e) Optimized Intermediate code.

$\text{temp}_1 = a * b$

$\text{temp}_2 = b * c$

$P = \text{temp}_1 + \text{temp}_2 * 2$

f) Target code (Example).

LOAD a

MUL b

STORE R1

LOAD b

MUL c

MUL 2

ADD R1

STORE P

Regular Expression:-

a) $(\epsilon + aa)^*$

{ ϵ (empty string)

aa,

aaaa,

aaaaaa,

aaaaaaaa }

b) $(a^* + b^*)^* abb$

All strings over {a,b} that end with "abb".

{ abb,

aabb,

babb,

aaabb,

bbabb, }

c) $(a^*b^*)^* aba (a^* + b^*)^*$ (All string that contain "aba".

{ aba,

aaba,

abaa,

babab,

aaababb }

d) $(a+ab)^*$

strings formed using a or ab repeatedly.

{ ϵ , a, ab, aa, abab... }

e) $(aba)^* + (a^*b^*)^*$

String that are either repetitions of "aba" or belong to a^*b^*

{ ϵ , aba, abaaba, aaabbb, aab }

3) (i) Number of Tokens.

Program:-

main()

```
{ int a=10;
  char b = 'abc';
  int c=30;
  char d = "xyz";
  int m=40.5;
```

}
Token Total count is 30

(ii) Lexical Errors:-

Program:-

void main()

```
{ int x=10, y=20;
  char *a;
  a = &x;
  x = 1xab;
```

}

Errors:-

- 1) 1xab is an invalid numeric literal.
- 2) a = &x; assigns an integer address to a character pointer.
- 3) Lexical analyzer reports 1xab as an invalid token.

4) Regular Expressions:-

(i) Even number of 0's.

$(1^*01^*01^*)^*1^*$

(ii) At least one 1

$(0|1)^*1(0|1)^*$

(iii) odd numbers of 0's and odd numbers of 1's.

$1^*(01^*01^*)^*01^*(10^*10^*)^*10^*$

(iv) starts with 1 and ends with 0

$1(0|1)^*0$

(v) No consecutive 1's

$(0|10)^*1?$

Lexical Analyzer.

Input:-

```
int var123 = 12345;  
float var456 = 123.45;  
if (var123 == var456)  
    { return;  
    }
```

a) Number of Tokens:-

total Token = 20.

b) Token Categories:-

Keywords	Identifiers	Operators	Separators
int	var123	=	;
float	var456	=	;
if	var123	==	(
return	var456		)
Count = 4	Count = 4	Count = 3	Count = 7

Integer literal	Floating literal
12345	123.45
Count = 1	Count = 1.

c) FSM state Transitions-

Approximate Transitions;

Keywords → 20

Identifiers → 28

operators → 4

separators → 7

Integer literal → 5

Floating literal → 7.

71